Civic Security

Informe de avance del MVP - backend, frontend y base de datos

Curso ET201, Formación de Empresas de Base Tecnológica I - Universidad Católica San Pablo
Integrantes: *Anthony Briceño, Maycol Canaveri y Marcelo Torres* | 5 de noviembre de 2026

Índice

1 Resumen 1
2 Estado del MVP frente al alcance 1
3 Base de datos 1
4 Backend 1
5 Frontend (aplicación móvil Android) 4
6 Diagramas UML 5
7 Pruebas y seguridad 5
8 Hallazgos de la prueba en tablet 5
9 Plan para la próxima semana 8
A Código representativo 8

1. Resumen

Civic Security es una red ciudadana para reportar incidentes de seguridad de forma anónima y validarlos de forma comunitaria en Arequipa. El MVP funciona hoy de punta a punta para **ver reportes en un mapa y enviar un reporte anónimo**; la **validación comunitaria**, diferencial del producto, está diseñada y queda para las siguientes iteraciones. Repositorio público: `https://github.com/aaabriceno/Proyecto-MVP-Civic-Security`.

- **Funciona hoy:** mapa centrado en la ubicación del usuario con los reportes del servidor, filtros por tipo, detalle y envío de un reporte anónimo (tipo, ubicación, descripción).
- **Anonimato por diseño:** el reporte no tiene campo de usuario, no se guarda la IP con él y se rechazan datos personales (DNI, teléfonos, correos, enlaces) en la descripción.
- **Calidad y seguridad:** 61 pruebas del backend y 120 de la app en verde; análisis estático y auditoría de dependencias sin hallazgos; revisión con ataques reales al servidor y 12 hallazgos corregidos.

2. Estado del MVP frente al alcance

Funcionalidad	Estado	Nota
Base de datos con PostGIS y datos de prueba	**Hecho**	Tabla `incidentes` con índice espacial
API para crear reportes y API del mapa	**Hecho**	`POST /v1/incidents`, `GET /v1/map/snapshot`
Canal en tiempo real en el backend	**Hecho**	La app todavía no lo escucha
Mapa con ubicación del usuario, pines por tipo, filtros y detalle	**Hecho**	Frame 05 de Figma (parcial)
Formulario de reporte anónimo con 6 tipos	**Hecho**	Frames 06, 06c y 07 de Figma
Seguridad básica de la API y de la app	**Hecho**	Sección 7
Pines nuevos en tiempo real en la app	Pendiente	Backend listo; falta la app
Cuentas, inicio de sesión y verificación por DNI	Pendiente	Frames 01–04 y 10
Validación comunitaria (3 confirmaciones a < 1 km)	Pendiente	**Prioridad n.º 1 del equipo**
Mapa de calor, alertas por zona y foto con rostros difuminados	Pendiente	Frames 05b, 09, 06a y 06b

3. Base de datos

El motor es **PostgreSQL 17 con PostGIS 3.5** en un contenedor Docker (`infra/docker-compose.yml`) que solo acepta conexiones locales. PostGIS resuelve dentro de la base de datos, con índices, las consultas geográficas del producto: reportes del área visible, reportes a menos de 1 km (regla de votación) y el futuro mapa de calor. El diseño se documenta en tres niveles (conceptual, lógico y físico); **verde = implementado, gris = planificado**.

Decisiones de diseño: identificadores **UUID** (los reportes no se pueden enumerar, OWASP API1:2023); `geography(Point,4326)` (distancias reales en metros; PostGIS guarda (longitud, latitud) y la API expone `lat` y `lng` por separado); índice **GiST** sobre `ubicacion` (las búsquedas por rectángulo del mapa no recorren todas las filas); `idempotency_key` **único** (un reintento por mala señal no duplica el reporte); **ninguna columna identifica al usuario**; la verificación por DNI guardará solo un HMAC y no se guardan datos biométricos; el detalle se elimina a los 5 años.

Para la demostración hay 80 reportes **ficticios** (`backend/priv/repo/seeds.exs`), que pasan por las mismas reglas que la API, y consultas de ejemplo (`demo_consultas.sql`): reportes por tipo y estado, filtros de tiempo, reportes a menos de 1 km (`ST_DWithin`) y una agrupación en celdas de unos 500 m.

4. Backend

Es un **monolito modular** en Elixir con Phoenix: el código se divide en módulos con responsabilidades claras (para mantenerlo) pero se despliega como un solo servicio con una sola base de datos (para operarlo y que rinda).

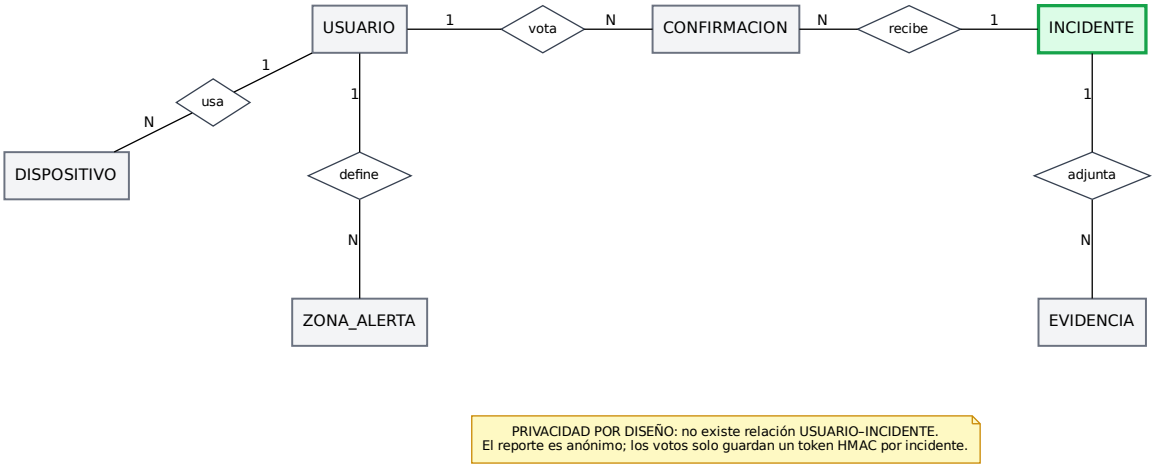


Figura 1: Modelo conceptual. Decisión de privacidad: **no existe relación entre USUARIO e INCIDENTE**.

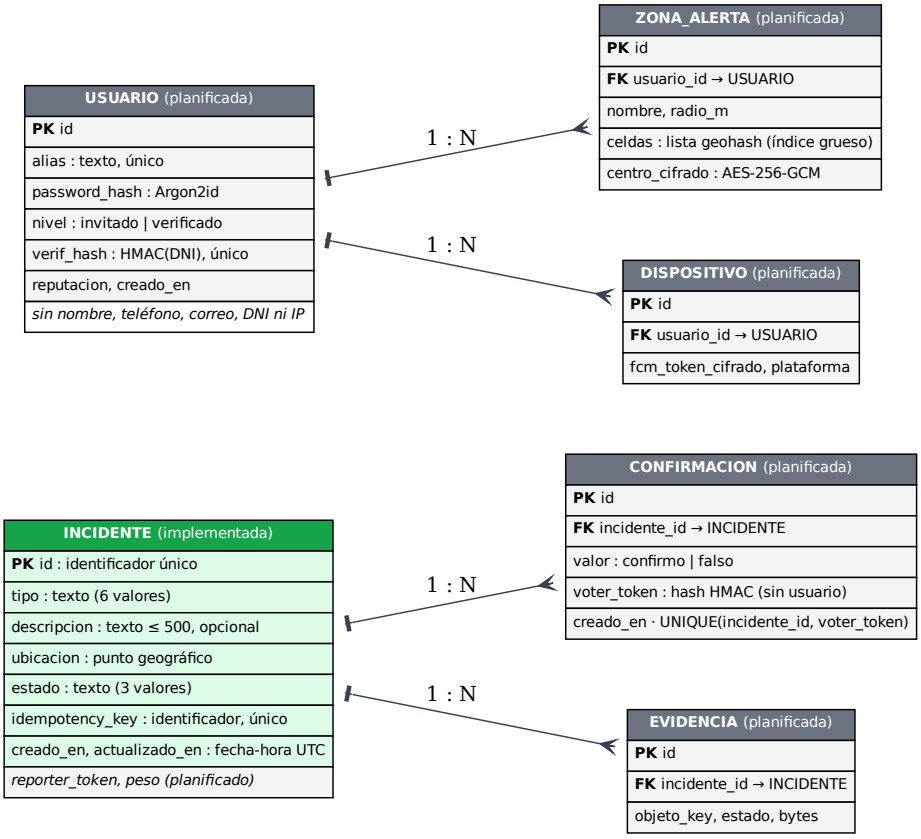


Figura 2: Modelo lógico: atributos, claves primarias (PK) y foráneas (FK).

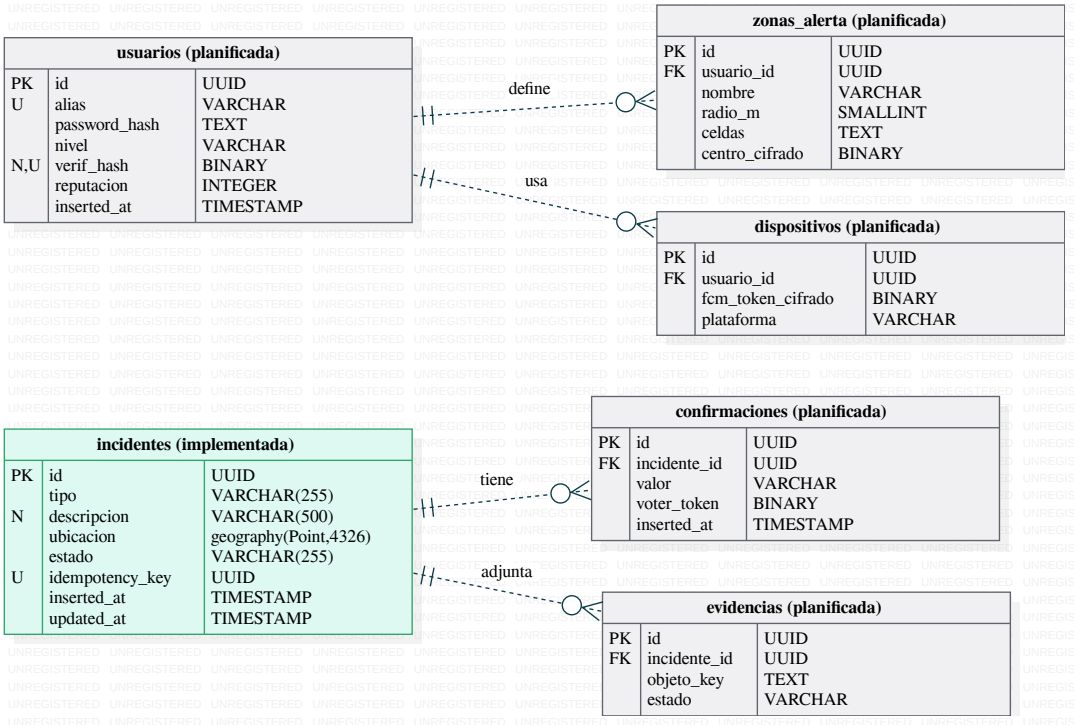


Figura 3: Modelo físico (StarUML): tipos de PostgreSQL. U = único, N = admite nulos. La tabla `incidentes` es la real (migración `20261027172454_create_incidentes.exs`).

La capa web (`lib/civic_web`) solo recibe peticiones; las reglas viven en el dominio (`lib/civic`). Así, las reglas que distinguen al producto — anonimato, área de Arequipa, no duplicados — se aplican en el servidor y no en la app, que cualquiera puede modificar.

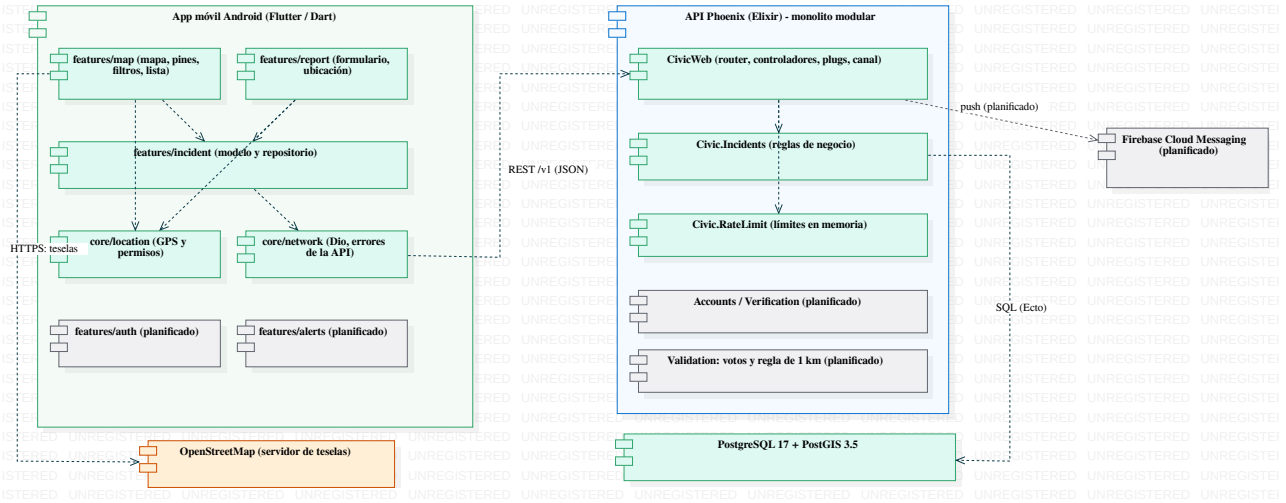


Figura 4: Diagrama de componentes (StarUML). Verde: implementado; gris: planificado.

4.1 API y reglas de negocio

Endpoint	Respuestas	Descripción
`GET /health`	200, 429	Comprueba que el servidor y la base de datos responden.
`POST /v1/incidents`	201, 200, 400, 409, 413, 415, 422, 429	Crea un reporte anónimo con el encabezado `idempotency-key`. 200 si se repite el mismo envío; 409 si la clave llega con otro contenido.
`GET /v1/map/snapshot`	200, 400, 429	Reportes de las últimas 24 h dentro del rectángulo `bbox=minLng,minLat,maxLng,maxLat` (máx. 500).
`WS /socket`	—	Tópico `mapa:arequipa`: emite `incidente_nuevo`; los clientes solo escuchan.

- Seis tipos de incidente; todo reporte nace **"Sin verificar"** y el estado no se puede enviar desde el cliente (lista blanca de campos: `tipo`, `descripcion`, `lat`, `lng`).
- Ubicación obligatoria y dentro de Arequipa; descripción opcional de hasta 500 caracteres, sin caracteres de control ni datos personales; envío **idempotente**.

Listing 1: Cadena de validaciones (`backend/lib/civic/incidents/incident.ex`).

```
def create_changeset(incident, attrs) do
  incident
```

```
  |> cast(attrs, [:tipo, :descripcion, :lat, :lng])
  |> validate_required([:tipo, :lat, :lng])
  |> update_change(:descripcion, &trim_or_nil/1)
  |> validate_length(:descripcion, max: 500)
  |> validate_texto_seguro(:descripcion)
  |> validate_sin_datos_personales(:descripcion)
  |> validate_arequipa()
  |> put_ubicacion()
  |> unique_constraint(:idempotency_key)
end
```

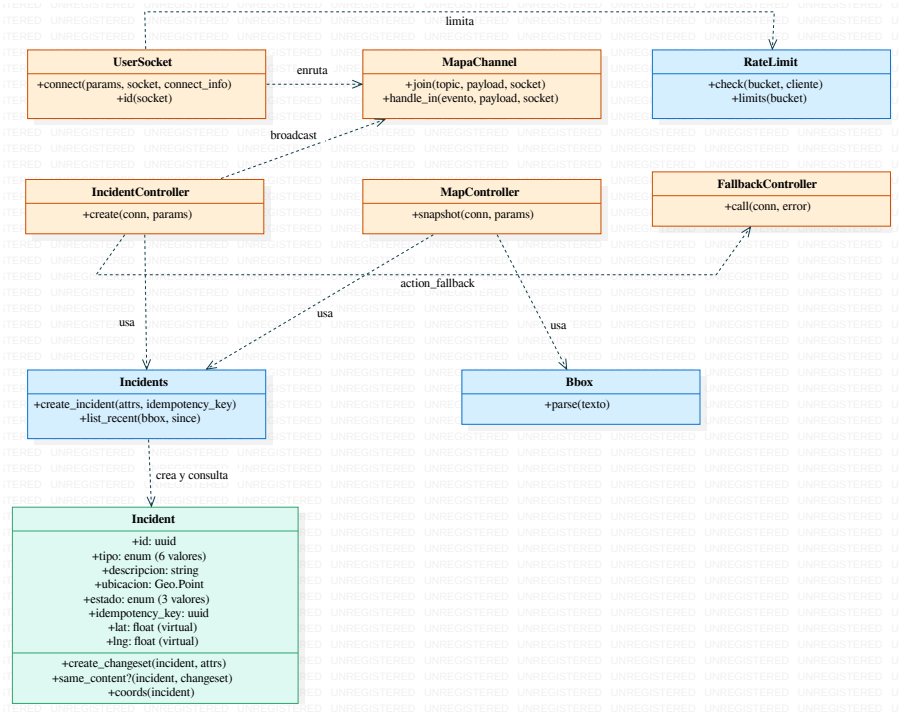


Figura 5: Diagrama de clases (módulos) del backend, con sus firmas reales (StarUML).

4.2 Evidencia: respuestas reales de la API

```
$ POST /v1/incidents (robo, lat -16.3963, lng -71.5340; el cliente intenta colar "estado":"verificado")
HTTP/1.1 201 Created {"data":{"id":"a2392f57-...","estado":"sin_verificar","tipo":"robo", ...}} <- se ignora
(mismo envio repetido con la misma clave) ......... HTTP 200
(sin encabezado idempotency-key) ................... HTTP 400 idempotency_key_required
(reporte en Lima) .................................. HTTP 422 "La ubicacion debe estar dentro de Arequipa"
(descripcion con un telefono) ...................... HTTP 422 "no escribas datos personales"
(cuerpo mayor a 8 KB) .............................. HTTP 413
```

5. Frontend (aplicación móvil Android)

App en **Flutter (Dart)**, probada en Android. Usa **Riverpod** (estado), **flutter_map** con OpenStreetMap (mapa), **geolocator** (ubicación) y **Dio** (API), organizada por funcionalidades (*feature-first*): `core/` (límites de Arequipa, ubicación, red, marcador del 105) y `features/` (`incident`: modelo y repositorio; `map`: mapa, pines, filtros, lista, detalle; `report`: formulario, ajuste de ubicación y confirmación). Se siguen los mockups de Figma (frames 05, 06, 06c y 07). Las capturas se tomaron en un emulador Android contra el backend real, con datos ficticios.

Comportamientos relevantes:

- **Ubicación:** permiso en primer plano; el mapa se centra en el usuario. Sin permiso, con el GPS apagado o fuera de Arequipa, muestra Arequipa completa con un aviso; no inventa una ubicación.
- **Carga de datos:** pide solo los reportes del *área visible*, tras 500 ms sin mover el mapa; una respuesta lenta no reemplaza a una más nueva y, ante un error de red, conserva los pines y ofrece “Reintentar”.
- **Envío seguro ante mala señal:** cada formulario genera una clave de idempotencia; reintentar lo mismo no duplica. El botón se deshabilita mientras envía.
- **Privacidad y transporte:** la ubicación del usuario no sale del teléfono; en producción la app exige `https`.

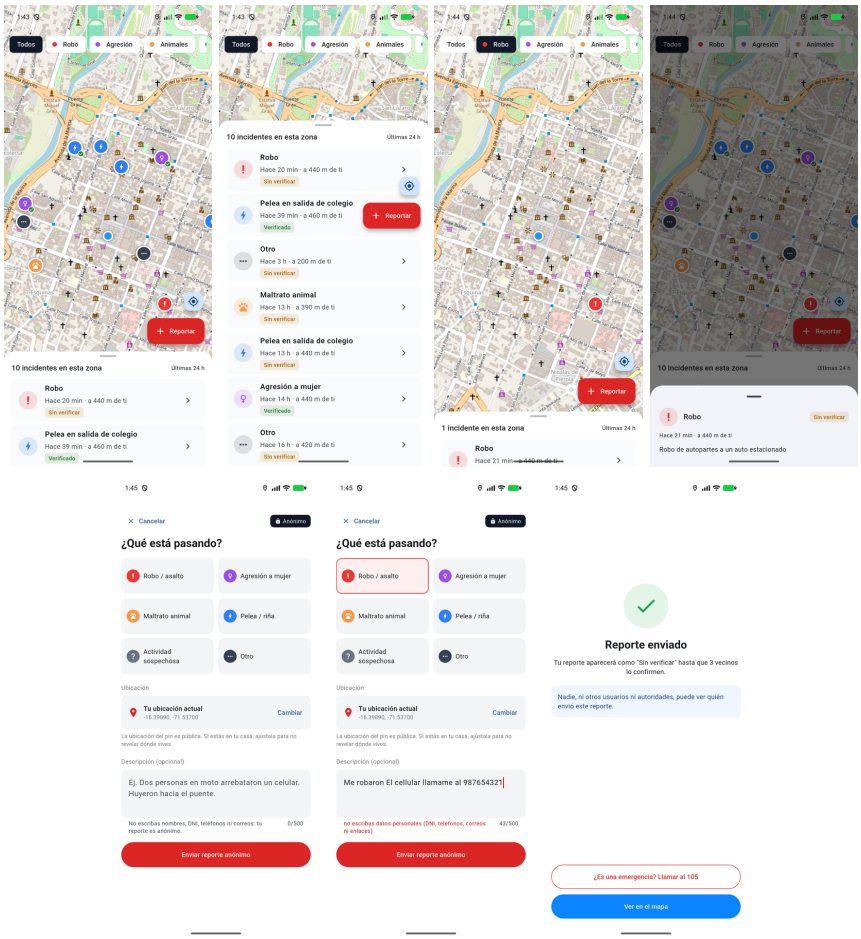


Figura 6: Arriba: mapa con pines por tipo, lista "N incidentes en esta zona" con hora y distancia, filtro por tipo (Robo) y detalle. Abajo: formulario con los seis tipos; el servidor bloquea un teléfono escrito en la descripción y la app lo explica; confirmación con "Llamar al 105".

6. Diagramas UML

Elaborados con StarUML. **Verde = implementado; gris punteado = planificado.**

7. Pruebas y seguridad

```
$ cd backend && mix test ........ 61 tests, 0 failures $ mix sobelow ........ sin hallazgos
$ mix deps.audit ................ No vulnerabilities found. $ cd app && flutter analyze ... No issues found!
$ flutter test .................. +120: All tests passed!
```

Se trabajó con *pruebas primero*: cada funcionalidad se describe con una prueba que falla y luego se implementa. Además se hizo una revisión actuando como atacante contra el servidor real, antes y después de cada corrección (informe completo en `docs/seguridad-revision-1.md`).

Pilar	Hallazgo	Corrección
Disponibilidad	Sin límite de solicitudes (150 reportes seguidos aceptados); cuerpos de 3 MB y formularios aceptados; marcos y conexiones WebSocket sin límite	Límites por cliente (429); solo JSON de hasta 8 KB (413, 415); marco máx. 4 KB y límite de conexiones
Integridad	La misma clave con otro contenido perdía el reporte en silencio; APK firmado con la clave de depuración	Responde 409; el release queda sin firmar si falta la clave propia
Confidencialidad	Servidor de desarrollo abierto a toda la red y a cualquier origen; sin cabeceras de seguridad	Escucha solo en localhost y verifica el origen; `nosniff`, CSP, `no-store`

Riesgos pendientes: la app usa el superusuario de la base de datos (solo desarrollo; antes del piloto se creará un rol limitado); no hay cuentas, así que cualquiera puede reportar (límites generosos por IP porque muchos alumnos comparten el wifi de la universidad); los reportes con ubicación simulada se tratarán con la validación comunitaria.

8. Hallazgos de la prueba en tablet

El 5 de noviembre se instaló la app en una tablet Android por USB contra el backend de la laptop. Funcionó de punta a punta; se investigaron dos observaciones:

1. **Lentitud al cargar y mover el mapa.** *Causa probable:* la prueba se hizo en modo **depuración** de Flutter, entre 3 y 10 veces más lento que la app real, en un dispositivo de gama media. *Descartado:* la descarga de teselas (`flutter_map` las guarda en disco y cancela las innecesarias). *Acción:* se habilitó el modo **profile** para medir

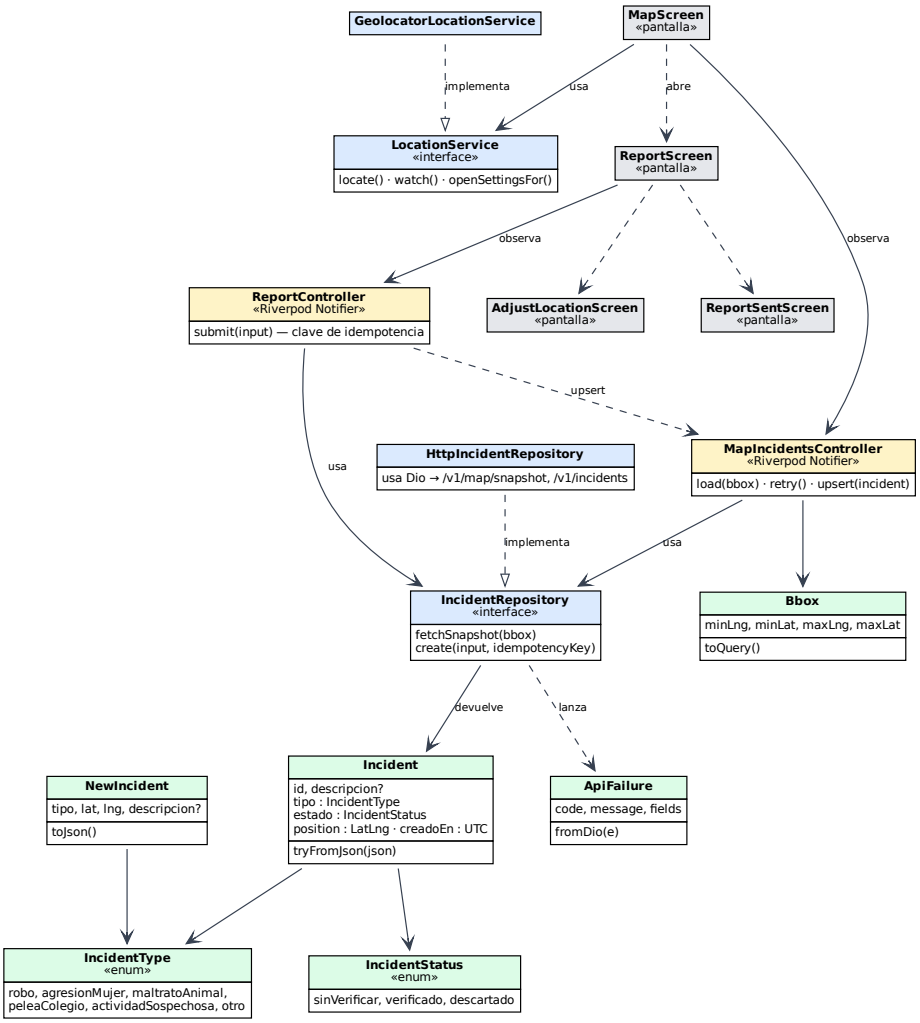


Figura 7: Diagrama de clases de la app: modelo, repositorio, controladores de estado y pantallas.

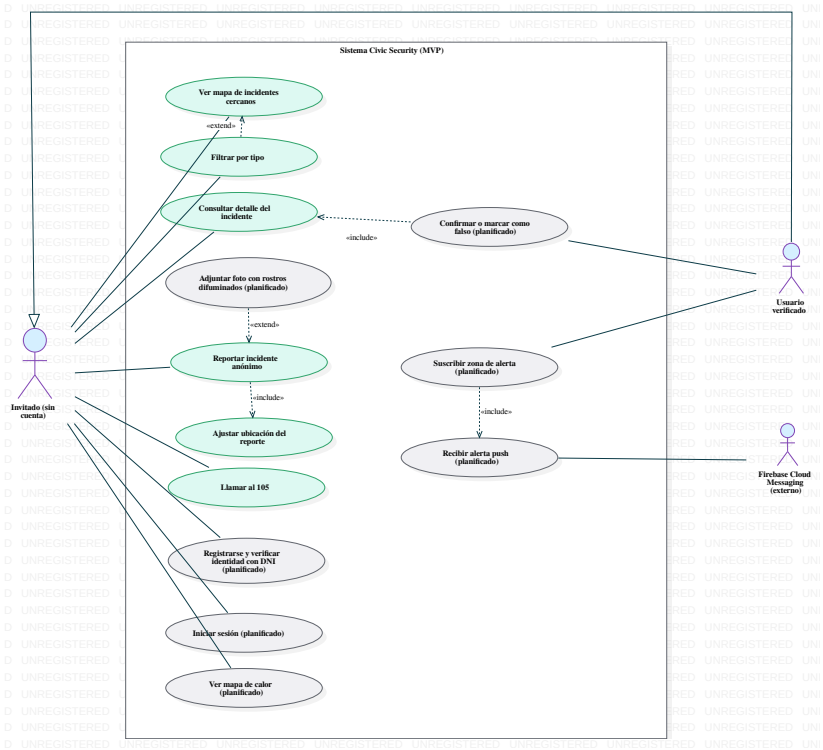


Figura 8: Casos de uso del MVP. El usuario verificado generaliza al invitado: puede, además, confirmar o desmentir reportes y suscribir zonas de alerta.

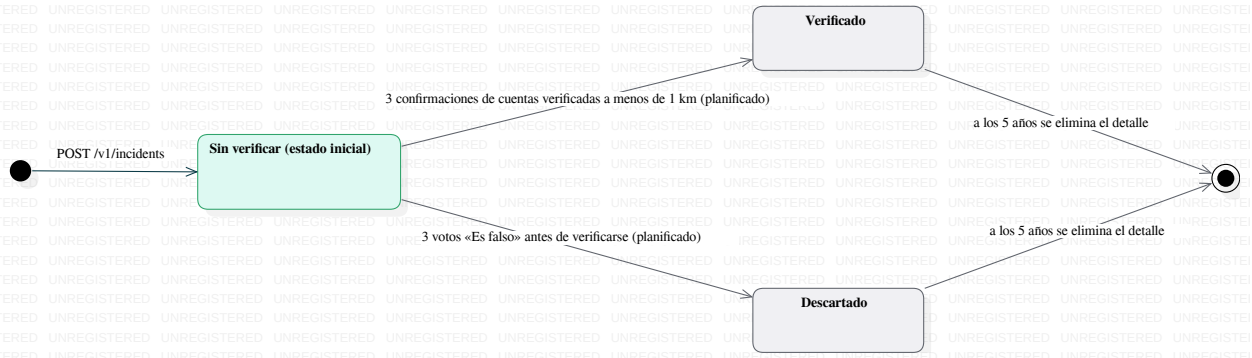


Figura 9: Estados del incidente. La columna `estado` ya admite los tres valores; las transiciones dependen de la validación comunitaria (iteración 2).

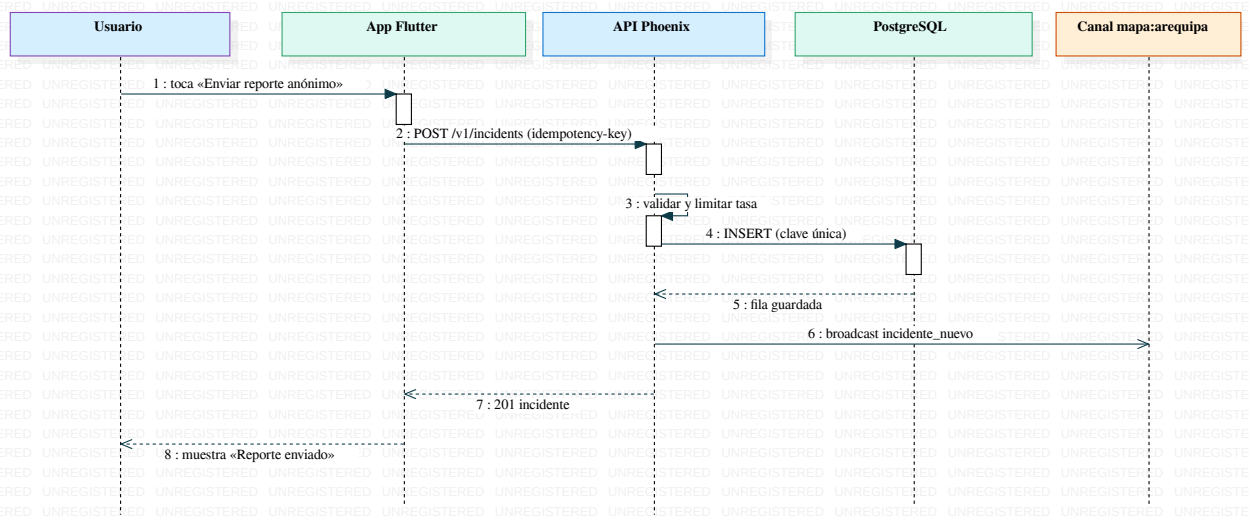


Figura 10: Secuencia: enviar un reporte anónimo. Si el servidor responde 200 (repetición con la misma clave) no se emite ningún evento.

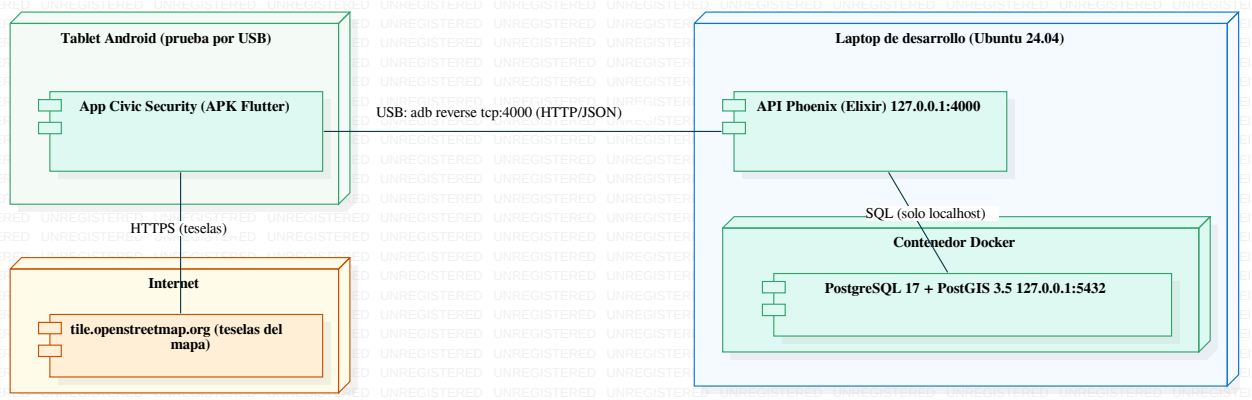


Figura 11: Despliegue del entorno de pruebas: la tablet se conecta por USB y `adb reverse` lleva su puerto 4000 al servidor de la laptop, que sigue cerrado a la red.

con el backend local; si la medición lo justifica, se limitará y agrupará los pines dibujados y se simplificarán sombras, para que la app funcione con fluidez en dispositivos de distintas capacidades.

2. **Distancia (“a 660 m de ti”) que parecía corta.** La fórmula de la app coincide con el cálculo exacto de PostGIS (`app/test/core/distance_test.dart`: 3 208 m Plaza de Armas–Alto Selva Alegre, 431 m a la UCSP). Es distancia en **línea recta**, no por calles, y puede diferir de lo que se aprecia si la **ubicación del dispositivo** es imprecisa (una tablet sin GPS ubica por Wi-Fi, con errores de cientos de metros). *Acción:* mostrar en el mapa un círculo con la precisión de la ubicación.

9. Plan para la próxima semana

Prio.	Actividad	Resultado esperado
1	**Integración continua** (GitHub Actions): pruebas, análisis estático, auditoría de dependencias y búsqueda de secretos en cada cambio	Cada cambio queda con marca verde o roja
1	**Tiempo real en la app**: escuchar `incidente_nuevo` y reconectar pidiendo el snapshot	Un reporte aparece en otro teléfono sin recargar
1	**Backend de cuentas**: sesión de invitado, registro con alias y contraseña (Argon2id), verificación por DNI con HMAC, límites por cuenta	Reportar y votar con cuenta
2	**Pantallas de Figma** 01–04 (bienvenida, login, registro, recuperación) conectadas a las cuentas	Flujo de registro funcional
2	**Rendimiento:** medir en modo *profile* en la tablet y optimizar; círculo de precisión de la ubicación	Mapa fluido en distintos dispositivos
3	**Validación comunitaria**: votos "Lo confirmo" / "Es falso", regla de 3 confirmaciones a menos de 1 km, transición de estados	Diferencial del producto funcionando

A. Código representativo

Listing 2: Migración de la tabla `incidentes` (`backend/priv/repo/migrations/20261027172454_create_incidentes.exs`).

```
create table(:incidentes, primary_key: false) do
  add :id, :binary_id, primary_key: true
  add :tipo, :string, null: false
  add :descripcion, :string, size: 500
  add :ubicacion, :"geography(Point,4326)", null: false
  add :estado, :string, null: false, default: "sin_verificar"
  add :idempotency_key, :binary_id, null: false
  timestamps(type: :utc_datetime_usec)
end
create unique_index(:incidentes, [:idempotency_key])
create index(:incidentes, [:ubicacion], using: :gist)
create index(:incidentes, [:inserted_at])
```

Listing 3: Consulta del mapa (`backend/lib/civic/incidents.ex`).

```
  def list_recent({min_lng, min_lat, max_lng, max_lat}, %DateTime{} = since) do
    Incident
    |> where([i], i.inserted_at > ^since)
    |> where(
      [i],
      fragment(
        "ST_Intersects(?, ST_MakeEnvelope(?, ?, ?, ?, 4326)::geography)",
        i.ubicacion,
        ^min_lng,
        ^min_lat,
        ^max_lng,
        ^max_lat
      )
    )
    |> order_by([i], desc: i.inserted_at)
    |> limit(^@max_resultados)
    |> Repo.all()
  end
```

Listing 4: Envío idempotente del reporte en la app (`app/lib/features/report/presentation/report_controller.dart`).

```
  Future<void> submit(NewIncident input) async {
    // Un doble toque, o enviar de nuevo algo ya enviado, no hace nada
    if (state.sending || state.created != null) return;

    if (input != _lastInput) {
      _key = _uuid.v4();
      _lastInput = input;
    }
    state = const ReportState(sending: true);

    try {
      final created = await ref
          .read(incidentRepositoryProvider)
          .create(input, idempotencyKey: _key);
      if (!ref.mounted) return;
      ref.read(mapIncidentsProvider.notifier).upsert(created);
      state = ReportState(created: created);
    } on ApiFailure catch (failure) {
      if (ref.mounted) {
        state = ReportState(failure: failure);
      }
    } on Object {
      if (ref.mounted) {
        state = const ReportState(failure: ApiFailure.unexpected());
      }
    }
  }
}
```